QUY HOẠCH ĐỘNG · 动态规划 · DYNAMIC PROGRAMMING

CHUYÊN ĐỀ BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

QUY HOẠCH ĐỘNG

CƠ BẢN & NÂNG CAO

qua 36 bài Luogu 洛谷

$$f[i][j] = f[i-1][j-1] + 1 \quad \text{nếu } a_i = b_j$$
$$f[i][j] = \max(f[i-1][j], f[i][j-1]) \quad \text{nếu khác}$$

		A	C	A	D	E	M	I	C
	0	0	0	**0**	0	0	0	0	0
D	0	0	0	0	**1**	**1**	1	1	1
Y	0	0	0	0	1	**1**	1	1	1
N	0	0	0	0	1	**1**	1	1	1
A	0	1	1	1	1	**1**	1	1	1
M	0	1	1	1	1	1	**2**	2	2
I	0	1	1	1	1	1	2	**3**	3
C	0	1	2	2	2	2	2	3	**4**

Bảng quy hoạch động của bài toán dãy con chung dài nhất giữa DYNAMIC và ACADEMIC. Ô đỏ đậm là các kí tự chung trên đường truy vết.

12 chương · 36 bài · 93 chương trình đã kiểm chứng và đo thời gian thật

01 Đệ quy có nhớ · 02 Dãy con · 03 Cái túi · 04 Biến thể cái túi · 05 DP trên đoạn · 06 DP trên cây · 07 Nhiều chiều · 08 Trạng thái bit · 09 DP chữ số · 10 Hàng đợi đơn điệu · 11 Bao lồi · 12 Kì vọng · Ma trận

LÊ THÀNH NAM

Lê Thành Nam

MỤC LỤC

MỤC LỤC 1
GIỚI THIỆU 3
Chương 1. Nhập môn: từ đệ quy đến quy hoạch động 6
Bài 1. Tam giác số 7
Bài 2. Tốt qua sông 12
Bài 3. Trượt tuyết 16
Chương 2. Dãy con: tăng dài nhất, chung dài nhất, khoảng cách chỉnh sửa 20
Bài 4. Hệ thống chặn tên lửa 21
Bài 5. Dãy con chung dài nhất của hai hoán vị (bài mẫu) 26
Bài 6. Khoảng cách chỉnh sửa 30
Chương 3. Cái túi 0/1 và cái túi không giới hạn 34
Bài 7. Hái thuốc 35
Bài 8. Hái thuốc không giới hạn 39
Bài 9. Hệ thống tiền tệ 43
Chương 4. Các biến thể của cái túi 47
Bài 10. Chọn lọc báu vật 48
Bài 11. Cái túi theo nhóm 53
Bài 12. Kế hoạch chi tiêu của Kim Minh 57
Chương 5. Quy hoạch động trên đoạn 61
Bài 13. Gộp đống sỏi 62
Bài 14. Tô màu 67
Bài 15. Tắt đèn đường 71
Chương 6. Quy hoạch động trên cây 75
Bài 16. Buổi tiệc không có sếp 76
Bài 17. Chọn môn học 80
Bài 18. Chọn gốc cho cây 85
Chương 7. Quy hoạch động nhiều chiều và kĩ thuật giảm chiều 89
Bài 19. Lấy số trên lưới 90
Bài 20. Chuyền giấy 94
Bài 21. Cờ rùa 98
Chương 8. Quy hoạch động trạng thái bit 101
Bài 22. Ruộng ngô 102
Bài 23. Các vua không ăn nhau 106
Bài 24. Bài toán người bán hàng 109
Chương 9. Quy hoạch động chữ số 113

Lê Thành Nam

Bài 25. Đếm chữ số 114
Bài 26. Số windy 117
Bài 27. Số điện thoại 120
Chương 10. Tối ưu bằng hàng đợi đơn điệu 123
Bài 28. Cirno qua sông 124
Bài 29. Cắt cỏ 128
Bài 30. Nhảy ô 131
Chương 11. Tối ưu bằng bao lồi 135
Bài 31. Xếp lịch xử lí theo đợt 136
Bài 32. Đóng gói đồ chơi 141
Bài 33. Đội đặc nhiệm 144
Chương 12. Quy hoạch động kì vọng và tăng tốc bằng ma trận 147
Bài 34. Chú ếch về nhà 148
Bài 35. Điểm OSU! 151
Bài 36. Dãy Fibonacci 154
PHỤ LỤC 157

Lê Thành Nam

GIỚI THIỆU

Cuốn sách này dành cho ai

Sách gồm 36 bài quy hoạch động chọn từ kho đề Luogu (洛谷, luogu.com.cn), nền tảng luyện thi tin học lớn nhất Trung Quốc. Các bài được xếp thành 12 chương, đi từ những ý tưởng nền tảng (đệ quy có nhớ, dãy con, cái túi) qua các mô hình kinh điển (quy hoạch động trên đoạn, trên cây, trạng thái bit, chữ số) đến các kĩ thuật tối ưu (hàng đợi đơn điệu, bao lồi, bất đẳng thức tứ giác, nhân ma trận) và quy hoạch động kì vọng.

Phần lớn các bài là đề thi chính thức (NOIP, NOI, APIO, USACO, POI, các kì thi chọn đội tuyển tỉnh của Trung Quốc như SCOI, HNOI, ZJOI, CQOI, SHOI) hoặc các bài kinh điển mà học sinh Trung Quốc nào cũng luyện. Bài dễ nhất phù hợp học sinh mới làm quen với quy hoạch động; bài khó nhất ngang tầm đề chọn đội tuyển quốc gia.

Mỗi bài gồm hai phần. **Đề bài** được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc, kèm tên gốc tiếng Trung và bảng subtask do sách bổ sung. **Hướng dẫn giải** đi qua nhiều tầng: lời giải ngây thơ, các tầng cải tiến, rồi mới đến lời giải tối ưu. Mỗi tầng đều có code chạy được, thời gian chạy đo thật và hình minh hoạ.

> **Về cách viết đề**
>
> Đề trong sách không phải bản dịch từng câu. Mỗi đề được viết lại theo cách diễn tự nhiên của người Việt, bỏ những chi tiết kể chuyện không ảnh hưởng đến lời giải. Mọi điều kiện, giới hạn, định dạng vào/ra và test ví dụ được giữ nguyên; mọi ví dụ đã được đối chiếu bằng cách chạy lời giải. Giới hạn thời gian ghi trong sách là giới hạn sách dùng để chấm các subtask, có thể khác giới hạn trên Luogu.

Cách giải

Hướng dẫn giải của mỗi bài chia thành các phần có dải màu riêng. Trong mỗi phần, lời giải đi từng bước nhỏ, và hầu hết các bước đều có hình minh hoạ.

Phần	Nội dung
HIỂU ĐỀ	Vét ra các trường hợp để hiểu chính xác yêu cầu; nêu các nhận xét đầu tiên
NGÂY THƠ	Tầng 1: ý tưởng tự nhiên nhất (thường là đệ quy vét cạn), code và độ phức tạp
BẾ TẮC	Thời gian chạy đo thật của tầng 1, chỉ ra nguyên nhân chậm, kết thúc bằng một *câu hỏi then chốt*
CẢI TIẾN 1, 2, ...	Các tầng trung gian: mỗi tầng sửa đúng một điểm yếu của tầng trước; có cả những tầng "nhanh nhưng sai" (tràn số, sai thứ tự vòng lặp, sai cận) để thấy lỗi thường gặp
LÓE SÁNG	Nhận xét quyết định dẫn tới lời giải tốt hơn, có lập luận và hình minh hoạ
TỐI ƯU	Code tối ưu, độ phức tạp và bảng so sánh tất cả các tầng trên mọi subtask
LƯU Ý	Những chỗ dễ sai và phần mở rộng

Ngoài ra, mỗi bài có ít nhất một hộp **Thiết kế quy hoạch động** màu vàng, tóm tắt bốn thành phần: trạng thái, công thức chuyển, cơ sở và đáp án. Đọc lướt các hộp này là cách ôn tập nhanh nhất.

Thời gian chạy trong sách được đo thật bằng trình biên dịch g++ 13 (tuỳ chọn -O2, chuẩn C++17) trên máy chủ Linux, tính cả thời gian đọc và ghi dữ liệu. Trên máy chấm Luogu các con số có thể khác đôi chút, nhưng tỉ lệ giữa các tầng là như nhau.

Subtask và các tầng lời giải

Mỗi bài có một bảng subtask: subtask đầu dành cho lời giải ngây thơ, các subtask giữa dành cho các tầng cải tiến, subtask cuối là ràng buộc gốc của Luogu. Với những bài mà đề gốc có ràng buộc nhỏ, sách mở rộng ràng buộc ở subtask cuối để lời giải tối ưu thật sự cần thiết; các subtask mở rộng được ghi rõ.

Lê Thành Nam

Cuối mỗi bài có bảng so sánh: mỗi tầng chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm đạt được. Ô đỏ "sai" nghĩa là chương trình cho kết quả sai; "quá bộ nhớ" nghĩa là chương trình dùng nhiều bộ nhớ hơn giới hạn ghi ở đầu bài.

Sử dụng sách

Với mỗi bài, nên đọc đề, tự nghĩ ít nhất 20–30 phút rồi mới đọc hướng dẫn, và dừng lại ở mỗi "Câu hỏi then chốt" để tự trả lời trước khi đọc tiếp. Mọi code trong sách đã được chạy với ví dụ của đề và đối chiếu chéo với nhau trên 120 test ngẫu nhiên mỗi bài (trừ những tầng được cố ý trình bày như một lời giải sai).

Nộp bài trên Luogu

- Đăng kí tài khoản tại luogu.com.cn (giao diện có thể chuyển sang tiếng Anh).
- Mở đề theo mã bài (ví dụ P1216) bằng đường dẫn trong sách, chọn 提交答案 (nộp bài), chọn ngôn ngữ C++14 hoặc C++17 (bật O2 nếu muốn), dán code và nộp.
- Chương trình đọc từ bàn phím (stdin) và ghi ra màn hình (stdout).
- Kết quả: AC (đúng), WA (sai đáp án), TLE (quá thời gian), MLE (quá bộ nhớ), RE (lỗi khi chạy). Luogu hiển thị kết quả từng test.

Bốn bước tư duy trước một bài quy hoạch động



Bốn bước xây dựng lời giải một bài quy hoạch động

- **Bước 1. Đọc ràng buộc.** Ràng buộc quyết định độ phức tạp được phép (bảng dưới), từ đó gợi ý kích thước trạng thái.
- **Bước 2. Viết vét cạn đúng.** Một hàm đệ quy thử mọi lựa chọn. Nó lấy điểm subtask nhỏ và là chương trình đối chiếu.
- **Bước 3. Tìm bài toán con lặp lại.** Hỏi: kết quả của lời gọi đệ quy có phụ thuộc vào "đường" đã đi tới đó không? Nếu không, những gì nó phụ thuộc chính là trạng thái.
- **Bước 4. Trạng thái, chuyển, thứ tự.** Ghi nhớ kết quả, tìm thứ tự tính, rồi tối ưu công thức chuyển (mảng cuộn, hàng đợi đơn điệu, bao lồi, ma trận…).

Ràng buộc n	Độ phức tạp tối đa	Dạng quy hoạch động thường gặp trong sách
≤ 20	$O(2^n \cdot n^2)$	trạng thái bit, người du lịch
≤ 100 – 500	$O(n^3)$	quy hoạch động trên đoạn, hai đường đi trên lưới
≤ 5 000	$O(n^2)$	dãy con, LCS, cái túi trên cây, DP khoảng có Knuth
≤ 10^5 – 10^6	O(n log n), O(n)	LIS nhị phân, DP trên cây, hàng đợi đơn điệu, bao lồi
≤ 10^{18}	O(log n), O(số chữ số)	nhân ma trận, quy hoạch động chữ số

Bố cục sách

Chương	Chủ đề	Bài (mã Luogu)
1	Nhập môn: từ đệ quy đến quy hoạch động	P1216, P1002, P1434
2	Dãy con: tăng dài nhất, chung dài nhất, khoảng cách chỉnh sửa	P1020, P1439, P2758

Lê Thanh Nam

Chương	Chủ đề	Bài (mã Luogu)
3	Cái túi 0/1 và cái túi không giới hạn	P1048, P1616, P5020
4	Các biến thể của cái túi	P1776, P1757, P1064
5	Quy hoạch động trên đoạn	P1880, P4170, P1220
6	Quy hoạch động trên cây	P1352, P2014, P3478
7	Quy hoạch động nhiều chiều và kĩ thuật giảm chiều	P1004, P1006, P1541
8	Quy hoạch động trạng thái bit	P1879, P1896, P1171
9	Quy hoạch động chữ số	P2602, P2657, P4124
10	Tối ưu bằng hàng đợi đơn điệu	P1725, P2627, P3957
11	Tối ưu bằng bao lồi	P2365, P3195, P3628
12	Quy hoạch động kì vọng và tăng tốc bằng ma trận	P4316, P1654, P1962

Lê Thành Nam

Chương 1. Nhập môn: từ đệ quy đến quy hoạch động

Trọng tâm chương

Mọi bài trong chương đều có một lời giải đệ quy rất tự nhiên nhưng chạy theo hàm mũ. Kĩ thuật chung là nhận ra **các bài toán con lặp lại**, lưu kết quả của chúng (đệ quy có nhớ), rồi tìm **thứ tự tính** để viết thành vòng lặp. Chương cũng rèn thói quen ước lượng độ lớn đáp án và bộ nhớ.

Kiến thức cần nhớ

- Quy hoạch động = chia bài toán thành bài toán con **gối nhau**, mỗi bài toán con giải **một lần**.
- Bốn câu hỏi khi thiết kế: **trạng thái** là gì, **chuyển** từ đâu, **cơ sở** là gì, **đáp án** nằm ở đâu.
- Đệ quy có nhớ: thêm mảng ghi kết quả vào hàm đệ quy; độ phức tạp = số trạng thái × chi phí mỗi trạng thái.
- Thứ tự tính phải đảm bảo trạng thái phụ thuộc được tính trước (theo hàng, theo độ cao, theo độ dài, …).
- Bài đếm số cách thường cần long long hoặc lấy modulo.

Các bài trong chương

Bài	Tên bài	Mã Luogu	Nguồn	Số tầng
1	Tam giác số [IOI 1994 / USACO1.5] 数字三角形 Number Triangles	P1216	IOI 1994 · USACO Training 1.5	3
2	Tốt qua sông [NOIP 2002 普及组] 过河卒	P1002	NOIP 2002 phổ cập	3
3	Trượt tuyết [SHOI2002] 滑雪	P1434	Thi chọn đội tuyển Thượng Hải (SHOI) 2002	3

Lê Thanh Nam

Bài 1. Tam giác số

Luogu P1216 – [IOI 1994 / USACO1.5] 数字三角形 Number Triangles

Nguồn: IOI 1994 · USACO Training 1.5 · Giới hạn thời gian sách dùng: 1,0 giây · Bộ nhớ: 128 MB
Đề gốc:https://www.luogu.com.cn/problem/P1216

Đề bài

Cho một tam giác số gồm r hàng, hàng thứ i có i số. Xuất phát từ số ở đỉnh, mỗi bước được đi xuống ô **ngay bên dưới bên trái** hoặc **ngay bên dưới bên phải**, cho đến khi tới một ô bất kì của hàng cuối.

Hãy tìm tổng lớn nhất của các số nằm trên đường đi.

Dữ liệu vào

Dòng đầu chứa số nguyên dương r là số hàng. r dòng tiếp theo, dòng thứ i chứa i số nguyên là các số trên hàng thứ i của tam giác.

Kết quả

Một số nguyên là tổng lớn nhất tìm được.

Ràng buộc

- 1 ≤ r ≤ 1000; mọi số trong tam giác là số nguyên trong đoạn [0, 100].

Ví dụ

Dữ liệu vào	Kết quả
5 7 3 8 8 1 0 2 7 4 4 4 5 2 6 5	30

Lê Thành Nam

Subtask

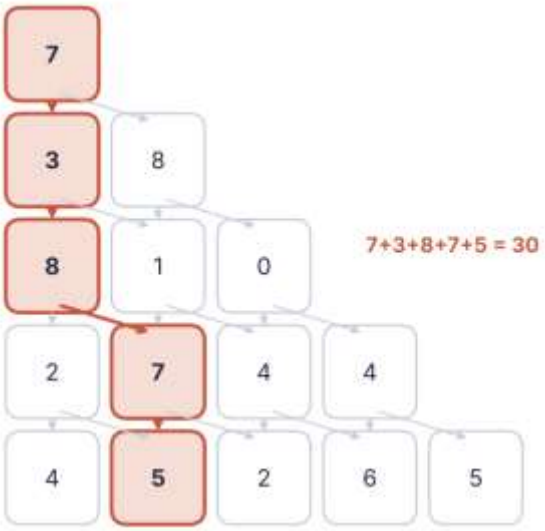
Các subtask dưới đây do sách bổ sung để luyện từng mức thuật toán; subtask cuối cùng là ràng buộc của đề gốc (hoặc mở rộng hơn đề gốc). Giới hạn thời gian 1,0 giây cho mọi subtask.

Subtask	Ràng buộc	Điểm
ST1	r ≤ 25	30%
ST2	r ≤ 1000 (gốc)	40%
ST3	r ≤ 5000, bộ nhớ 128 MB (mở rộng)	30%

Hướng dẫn giải

HIỂU ĐỀ Hiểu đúng đề bài

Dịch tam giác về bên trái để thấy rõ: từ ô (i, j) (hàng i, cột j, đánh số từ 0) chỉ có thể đi xuống (i + 1, j) hoặc (i + 1, j + 1). Mỗi đường đi gồm đúng r ô, mỗi hàng một ô. Với ví dụ, đường tốt nhất là 7 → 3 → 8 → 7 → 5, tổng 30.



Hình 1.1. *Tam giác của ví dụ, căn trái. Đường tô đỏ có tổng 30.*

Mỗi bước có 2 lựa chọn, nên có tất cả 2^{r-1} đường đi. Với r = 1000, con số này lớn hơn số nguyên tử trong vũ trụ.

NGÂY THƠ Ý tưởng ngây thơ: thử mọi đường bằng đệ quy

Gọi best(i, j) là tổng lớn nhất khi xuất phát từ ô (i, j) đi xuống đáy. Đứng ở (i, j), ta cộng số của ô mình với kết quả tốt hơn trong hai ô bên dưới:

best(i, j) = a[i][j] + max(best(i + 1, j), best(i + 1, j + 1)), và ở hàng cuối thì best = a[i][j].

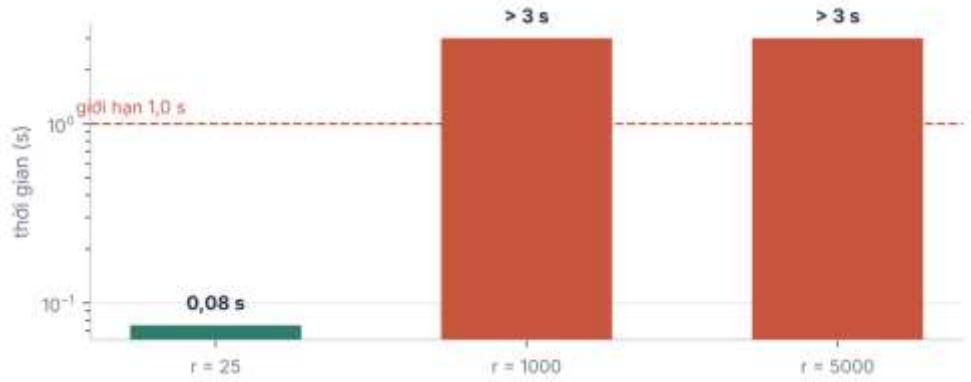
Code. Tầng 1 – Đệ quy thử mọi đường đi

```
#include <bits/stdc++.h>
using namespace std;
int n;
vector<vector<int>> a;
// tổng lớn nhất đi từ ô (i, j) xuống đáy: thử cả hai hướng
int best(int i, int j) {
    if (i == n - 1) return a[i][j];
    return a[i][j] + max(best(i + 1, j), best(i + 1, j + 1));
}
int main() {
    scanf("%d", &n);
    a.assign(n, vector<int>());
    for (int i = 0; i < n; i++) {
        a[i].resize(i + 1);
        for (int j = 0; j <= i; j++) scanf("%d", &a[i][j]);
    }
    printf("%d\n", best(0, 0));
}
```

Lời giải đúng nhưng mỗi lời gọi sinh ra hai lời gọi con, nên tổng số lời gọi là $2^r - 1$.

Lê Thành Nam

BẾ TẮC Bế tắc: r = 1000



Hình 1.2. *Tầng 1 trên các nhóm test (thời gian đo thật, trục log)*

Vì sao lại nhiều đến vậy, trong khi tam giác chỉ có r(r + 1)/2 ô? Hãy đếm xem mỗi ô được gọi bao nhiêu lần.



Hình 1.3. *Số lần best(i, j) được gọi trong đệ quy ngây thơ (r = 6). Đó chính là tam giác Pascal: ô (i, j) bị gọi C(i, j) lần.*

Ô (i, j) bị gọi đúng bằng số đường đi từ đỉnh tới nó, tức C(i, j) lần. Ở hàng thứ 25 đã có ô bị tính lại hơn 2,7 triệu lần, dù kết quả lần nào cũng giống nhau.

> **Câu hỏi then chốt**
>
> best(i, j) chỉ phụ thuộc vào (i, j), không phụ thuộc vào việc ta đi tới (i, j) bằng đường nào. Vậy có cần tính nó nhiều hơn một lần không?

CẢI TIẾN 1 Cải tiến 1: đệ quy có nhớ

Thêm mảng f[i][j], ban đầu bằng −1 (nghĩa là "chưa tính"). Lần đầu tính xong best(i, j) thì ghi vào f[i][j]; những lần sau chỉ cần đọc ra. Kĩ thuật này gọi là **đệ quy có nhớ** (记忆化搜索).

Thiết kế quy hoạch động	
Trạng thái	f[i][j] = tổng lớn nhất đi từ ô (i, j) xuống đáy
Chuyển	f[i][j] = a[i][j] + max(f[i+1][j], f[i+1][j+1])
Cơ sở	hàng cuối: f[r−1][j] = a[r−1][j]
Đáp án	f[0][0]

Lê Thành Nam

Code. Tầng 2 – Đệ quy có nhớ (mảng 2 chiều)

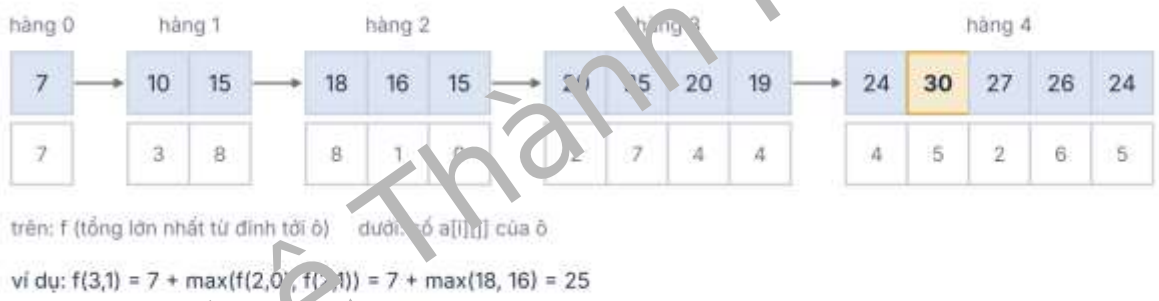
```
#include <bits/stdc++.h>
using namespace std;
const int N = 5005;
int n, a[N][N], f[N][N];   // f[i][j] = -1: chưa tính
int best(int i, int j) {
    if (f[i][j] != -1) return f[i][j];          // đã tính rồi: trả ngay
    if (i == n - 1) return f[i][j] = a[i][j];
    return f[i][j] = a[i][j] + max(best(i + 1, j), best(i + 1, j + 1));
}
int main() {
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j <= i; j++) scanf("%d", &a[i][j]);
    memset(f, -1, sizeof f);
    printf("%d\n", best(0, 0));
}
```

Mỗi ô được tính đúng một lần, mỗi lần O(1): tổng $O(r^2)$. Với r = 1000 chương trình chạy tức thì. Ở ST3 (r = 5000), hai mảng 5005 × 5005 số int chiếm khoảng 200 MB, vượt giới hạn 128 MB.

LÓE SÁNG Lóe sáng: tính theo thứ tự, và chỉ giữ một hàng

Đệ quy có nhớ tự tìm ra thứ tự tính. Ta cũng có thể chủ động tính theo thứ tự hàng: gọi f[i][j] là tổng lớn nhất **từ đỉnh đi xuống** ô (i, j). Ô (i, j) chỉ có thể đến từ (i − 1, j − 1) hoặc (i − 1, j), đều nằm ở hàng ngay trên. Do đó:

- Tính xong hàng i − 1 là đủ để tính hàng i, và mọi hàng trước đó có thể bỏ đi.
- Không cần lưu cả tam giác: đọc hàng nào, xử lí hàng đó.



Hình 1.4. Giá trị f theo từng hàng của ví dụ. Mỗi ô lấy số lớn hơn trong hai ô phía trên rồi cộng số của mình. Đáp án là số lớn nhất ở hàng cuối.

Chỉ giữ một mảng f dài r. Khi cập nhật hàng i, duyệt j **giảm dần**: lúc ghi đè f[j] thì f[j − 1] vẫn còn là giá trị của hàng i − 1.

Thiết kế quy hoạch động – từ trên xuống	
Trạng thái	f[j] = tổng lớn nhất từ đỉnh tới ô (i, j) của hàng đang xét
Chuyển	f[j] ← a[i][j] + max(f[j−1], f[j]), duyệt j từ i về 0
Cơ sở	f[0] = a[0][0]
Đáp án	max f[j] ở hàng cuối

TỐI ƯU Lời giải tối ưu

Code. Tầng 3 – Quy hoạch động từ trên xuống, một hàng

```
#include <bits/stdc++.h>
using namespace std;
// đọc nhanh số nguyên không âm
static char buf[1 << 25];
int bl = 0, bp = 0;
inline int gc() {
```

Lê Thành Nam